

Quiz 4 Solutions

```
if (!requireNamespace("caret", quietly = TRUE)) install.packages("caret")
if (!requireNamespace("rpart", quietly = TRUE)) install.packages("rpart")
if (!requireNamespace("rattle", quietly = TRUE)) install.packages("rattle")
if (!requireNamespace("randomForest", quietly = TRUE)) install.packages("randomForest")

library(caret)
library(rpart)
library(rattle)
library(randomForest)
```

Task 1. Regression with the Ames Housing Data

Q1. Clean the data and split into training/testing sets

```
ames <- read.csv("Ames_Housing_Data.csv", stringsAsFactors = TRUE)

# Remove observations with missing values
ames_clean <- na.omit(ames)
cat("Number of observations remaining after removing missing values:", nrow(ames_clean), "\n")
```

```
## Number of observations remaining after removing missing values: 1460
```

```
set.seed(123)
train_index_ames <- createDataPartition(ames_clean$SalePrice, p = 0.75, list = FALSE)
ames_train <- ames_clean[train_index_ames, ]
ames_test <- ames_clean[-train_index_ames, ]

cat("Training sample size:", nrow(ames_train), "\n")
```

```
## Training sample size: 1097
```

```
cat("Testing sample size:", nrow(ames_test), "\n")
```

```
## Testing sample size: 363
```

Q2. Prune a CART tree with 10-fold CV and compute Test RMSE

```

set.seed(123)
ames_cart <- train(
  SalePrice ~ .,
  data = ames_train,
  method = "rpart",
  trControl = trainControl(method = "cv", number = 10),
  tuneLength = 20
)

```

```

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

```

```

best_cp_ames <- ames_cart$bestTune$cp
cat("Best CP for Ames CART:", best_cp_ames, "\n")

```

```

## Best CP for Ames CART: 0.0027912

```

```

ames_cart$finalModel

```

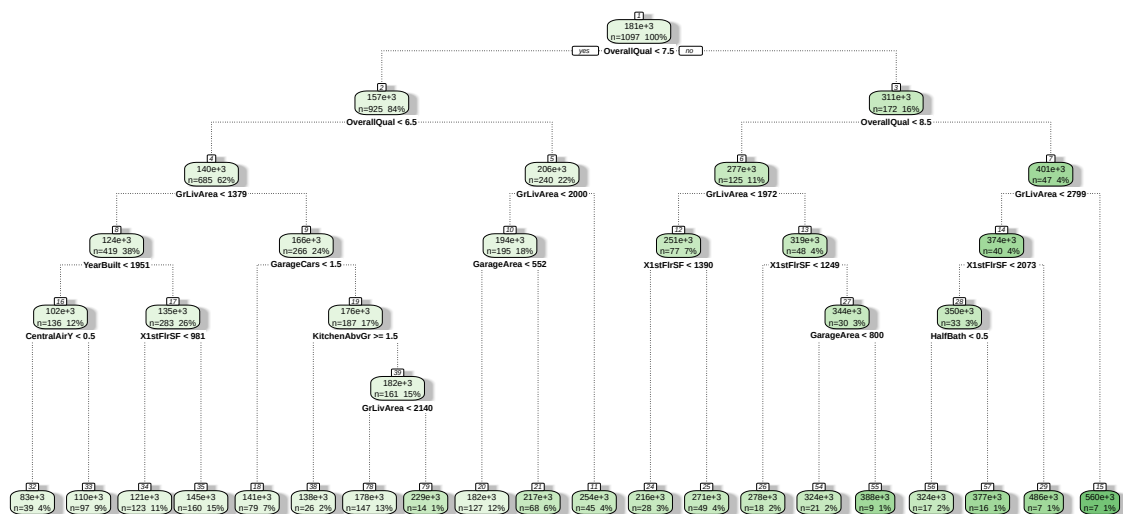
```

## n= 1097
##
## node), split, n, deviance, yval
##      * denotes terminal node
##
## 1) root 1097 7.207687e+12 181273.40
##    2) OverallQual< 7.5 925 2.225302e+12 157161.00
##      4) OverallQual< 6.5 685 1.012090e+12 140215.50
##        8) GrLivArea< 1378.5 419 3.634389e+11 124083.50
##          16) YearBuilt< 1950.5 136 9.052156e+10 102068.30
##            32) CentralAirY< 0.5 39 2.760795e+10 82660.64 *
##            33) CentralAirY>=0.5 97 4.231786e+10 109871.40 *
##          17) YearBuilt>=1950.5 283 1.753255e+11 134663.30
##            34) X1stFlrSF< 980.5 123 5.061599e+10 120945.80 *
##            35) X1stFlrSF>=980.5 160 8.377198e+10 145208.60 *
##          9) GrLivArea>=1378.5 266 3.678498e+11 165626.40
##            18) GarageCars< 1.5 79 6.637194e+10 141025.30 *
##            19) GarageCars>=1.5 187 2.334671e+11 176019.40
##              38) KitchenAbvGr>=1.5 26 1.730874e+10 137873.20 *
##              39) KitchenAbvGr< 1.5 161 1.722152e+11 182179.70
##                78) GrLivArea< 2140 147 1.168406e+11 177714.00 *
##                79) GrLivArea>=2140 14 2.166217e+10 229069.30 *
##          5) OverallQual>=6.5 240 4.551087e+11 205526.20
##            10) GrLivArea< 1999.5 195 2.449827e+11 194277.80
##              20) GarageArea< 552 127 1.134244e+11 182266.40 *
##              21) GarageArea>=552 68 7.901490e+10 216710.90 *
##            11) GrLivArea>=1999.5 45 7.853876e+10 254269.20 *
##          3) OverallQual>=7.5 172 1.552332e+12 310947.70
##            6) OverallQual< 8.5 125 5.305813e+11 276992.60
##              12) GrLivArea< 1971.5 77 1.706257e+11 250836.90
##                24) X1stFlrSF< 1389.5 28 3.624184e+10 215588.80 *
##                25) X1stFlrSF>=1389.5 49 7.971679e+10 270978.80 *

```

```
##      13) GrLivArea>=1971.5 48 2.227754e+11 318950.70
##      26) X1stFlrSF< 1249 18 4.506533e+10 277984.80 *
##      27) X1stFlrSF>=1249 30 1.293779e+11 343530.20
##      54) GarageArea< 799.5 21 4.274787e+10 324416.30 *
##      55) GarageArea>=799.5 9 6.105622e+10 388129.20 *
##      7) OverallQual>=8.5 47 4.943397e+11 401253.60
##      14) GrLivArea< 2798.5 40 2.173707e+11 373515.60
##      28) X1stFlrSF< 2072.5 33 7.365730e+10 349707.10
##      56) HalfBath< 0.5 17 2.002336e+10 323564.70 *
##      57) HalfBath>=0.5 16 2.967143e+10 377483.30 *
##      29) X1stFlrSF>=2072.5 7 3.682210e+10 485756.00 *
##      15) GrLivArea>=2798.5 7 7.033148e+10 559756.30 *
```

```
fancyRpartPlot(ames_cart$finalModel)
```



Rattle 2026-Apr-05 15:26:25 ian_wang

```
ames_cart_pred <- predict(ames_cart, newdata = ames_test)
ames_cart_rmse <- sqrt(mean((ames_cart_pred - ames_test$SalePrice)^2))
cat("Ames CART Test RMSE:", round(ames_cart_rmse, 4), "\n")
```

```
## Ames CART Test RMSE: 45005.82
```

Q3. Tune RF with tuneRF, fit final randomForest, plot importance, and compute Test RMSE

```
set.seed(123)
ames_tune <- tuneRF(
  x = subset(ames_train, select = -SalePrice),
  y = ames_train$SalePrice,
  stepFactor = 1.5,
  improve = 0.01,
  trace = TRUE,
  plot = FALSE
)
```

```
## mtry = 6   OOB error = 796485391
## Searching left ...
## mtry = 4   OOB error = 822792807
## -0.03302938 0.01
## Searching right ...
## mtry = 9   OOB error = 785541801
## 0.01373985 0.01
## mtry = 13  OOB error = 844601288
## -0.07518312 0.01
```

```
ames_tune
```

```
##      mtry OOBError
## 4         4 822792807
## 6         6 796485391
## 9         9 785541801
## 13        13 844601288
```

```
best_mtry_ames <- ames_tune[which.min(ames_tune[, 2]), 1]
cat("Best mtry for Ames RF:", best_mtry_ames, "\n")
```

```
## Best mtry for Ames RF: 9
```

```
set.seed(123)
ames_rf <- randomForest(
  SalePrice ~ .,
  data = ames_train,
  mtry = best_mtry_ames,
  importance = TRUE
)
```

```
ames_rf
```

```
##
```

```
## Call:
```

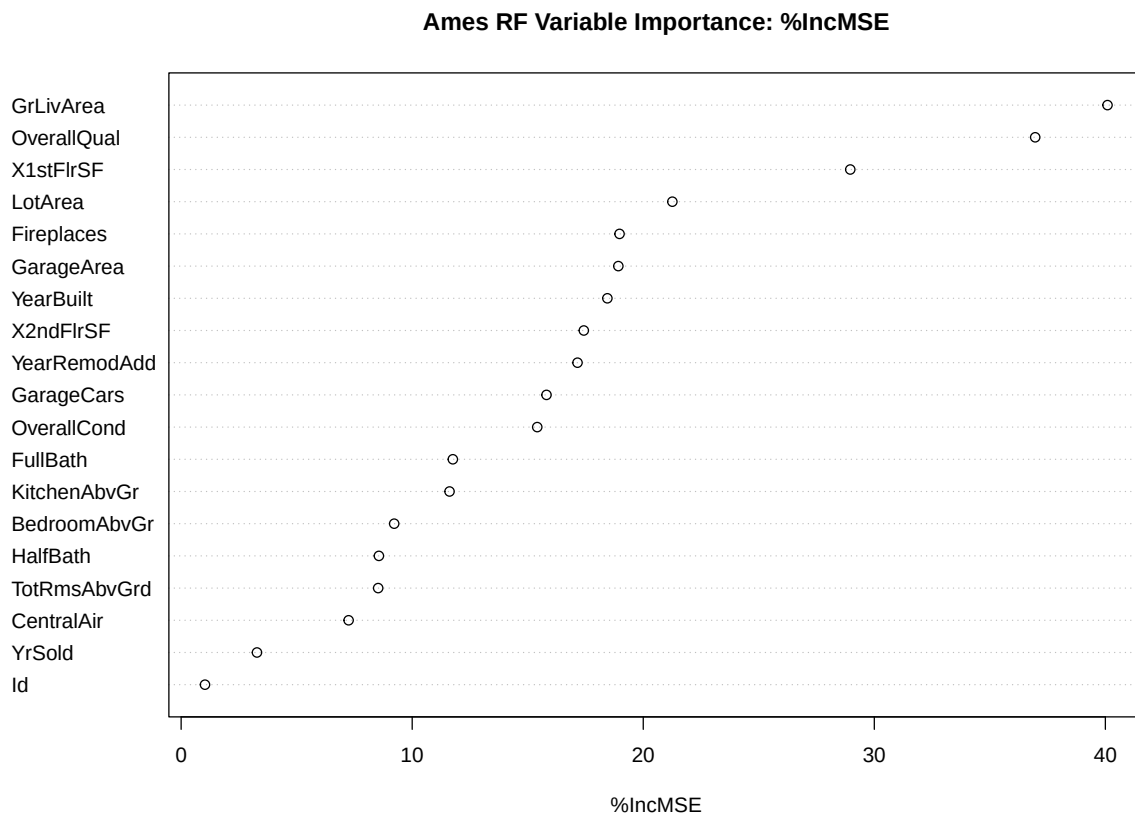
```
## randomForest(formula = SalePrice ~ ., data = ames_train, mtry = best_mtry_ames,
```

```
##                      Type of random forest: regression
```

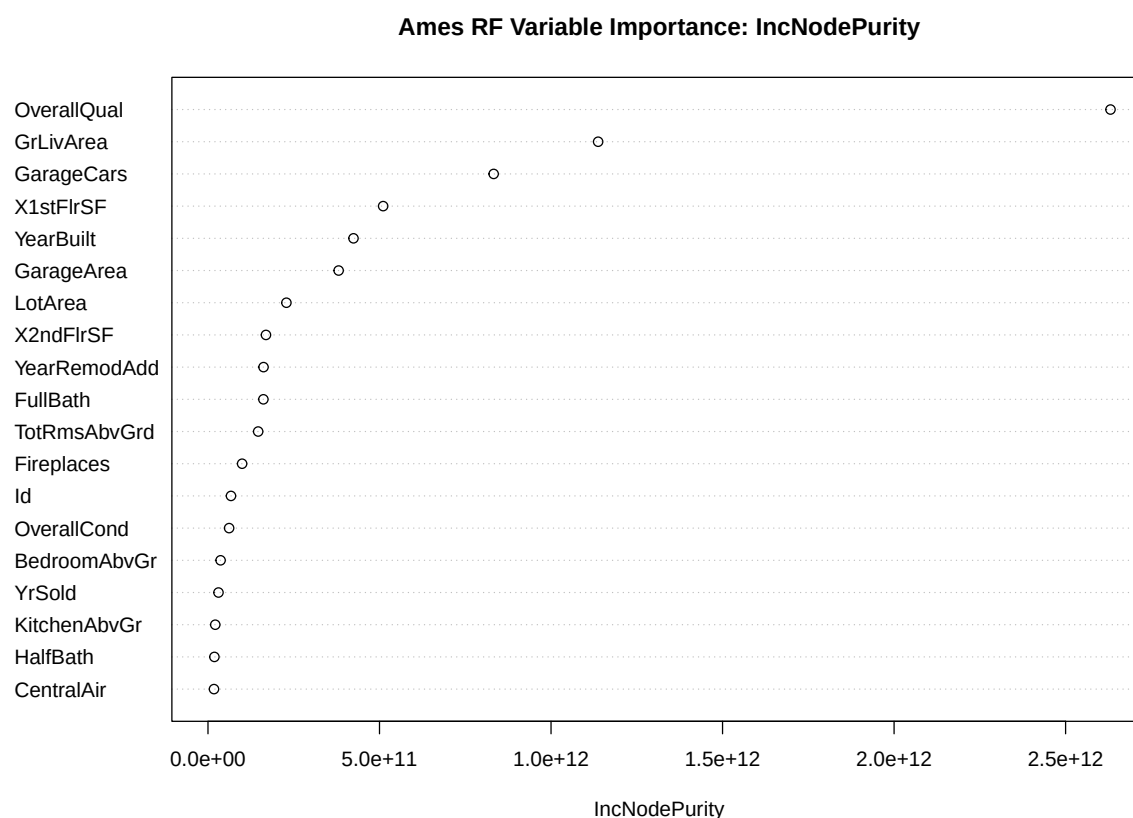
```
importance = T
```

```
##                               Number of trees: 500
## No. of variables tried at each split: 9
##
##           Mean of squared residuals: 732962470
##                               % Var explained: 88.84
```

```
randomForest::varImpPlot(
  ames_rf,
  type = 1,
  main = "Ames RF Variable Importance: %IncMSE"
)
```



```
randomForest::varImpPlot(
  ames_rf,
  type = 2,
  main = "Ames RF Variable Importance: IncNodePurity"
)
```



```
ames_rf_pred <- predict(ames_rf, newdata = ames_test)
ames_rf_rmse <- sqrt(mean((ames_rf_pred - ames_test$SalePrice)^2))
cat("Ames RF Test RMSE:", round(ames_rf_rmse, 4), "\n")
```

```
## Ames RF Test RMSE: 37323.27
```

Q4. Compare CART and RF using Test RMSE

```
cat("Ames CART Test RMSE:", round(ames_cart_rmse, 4), "\n")
```

```
## Ames CART Test RMSE: 45005.82
```

```
cat("Ames RF Test RMSE:", round(ames_rf_rmse, 4), "\n")
```

```
## Ames RF Test RMSE: 37323.27
```

The model with the lower Test RMSE (RF) performs better. In general, random forest often performs better than a single CART tree because it averages many trees, reduces variance, and captures more complex nonlinear patterns.

Task 2. Classification with the Titanic Data

Q5. Clean the data, recode variables, and split into training/testing sets

```
titanic <- read.csv("Titanic.csv", stringsAsFactors = FALSE)

# Remove specified variables
# Adjust PassengerId/PassengerID name if needed
if ("PassengerId" %in% names(titanic)) {
  titanic <- subset(titanic, select = -c(PassengerId, Name, Ticket, Cabin))
} else if ("PassengerID" %in% names(titanic)) {
  titanic <- subset(titanic, select = -c(PassengerID, Name, Ticket, Cabin))
}

# Remove missing values in Age first
if ("Age" %in% names(titanic)) {
  titanic <- titanic[!is.na(titanic$Age), ]
}

# Remove any remaining missing values so the models can run cleanly
# (for some Titanic files, Embarked may still have a small number of NAs)
titanic <- na.omit(titanic)

# Recode categorical variables
titanic$Survived <- factor(titanic$Survived, levels = c(0, 1))
titanic$Pclass <- as.factor(titanic$Pclass)

cat("Number of observations remaining after cleaning:", nrow(titanic), "\n")

## Number of observations remaining after cleaning: 714

set.seed(123)
train_index_titanic <- createDataPartition(titanic$Survived, p = 0.75, list = FALSE)
titanic_train <- titanic[train_index_titanic, ]
titanic_test <- titanic[-train_index_titanic, ]

cat("Training sample size:", nrow(titanic_train), "\n")

## Training sample size: 536

cat("Testing sample size:", nrow(titanic_test), "\n")

## Testing sample size: 178
```

Q6. Prune a CART tree with 10-fold CV and compute test sensitivity, specificity, and accuracy

```

set.seed(123)
titanic_cart <- train(
  Survived ~ .,
  data = titanic_train,
  method = "rpart",
  trControl = trainControl(method = "cv", number = 10),
  tuneLength = 20
)

best_cp_titanic <- titanic_cart$bestTune$cp
cat("Best CP for Titanic CART:", best_cp_titanic, "\n")

```

```
## Best CP for Titanic CART: 0.02341864
```

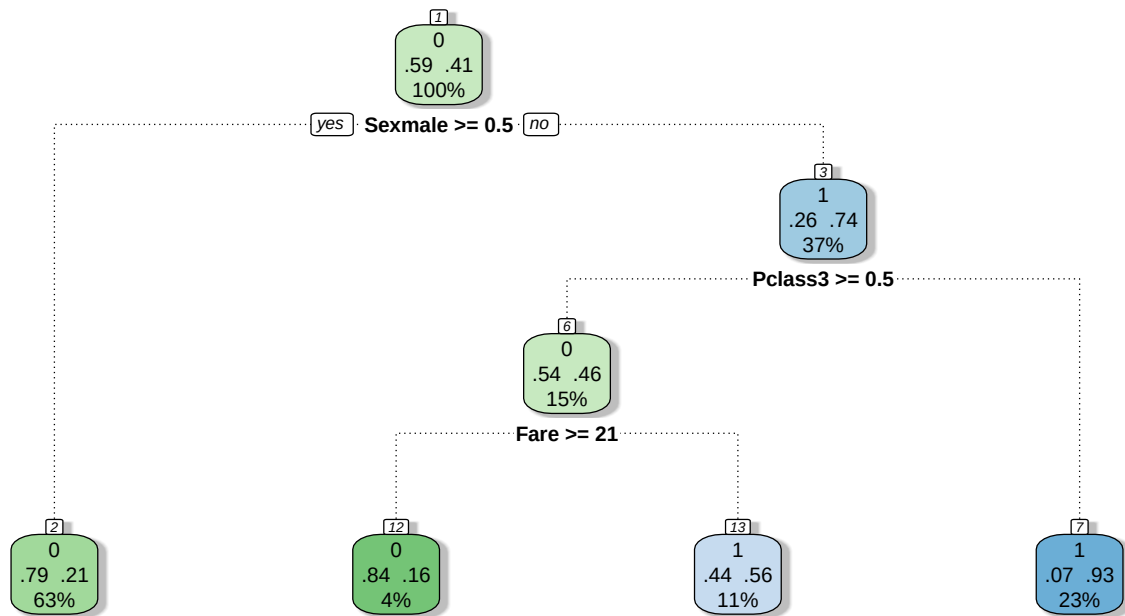
```
titanic_cart$finalModel
```

```

## n= 536
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
##  1) root 536 218 0 (0.59328358 0.40671642)
##    2) Sexmale>=0.5 337  70 0 (0.79228487 0.20771513) *
##    3) Sexmale< 0.5 199  51 1 (0.25628141 0.74371859)
##      6) Pclass3>=0.5 78  36 0 (0.53846154 0.46153846)
##        12) Fare>=20.8 19   3 0 (0.84210526 0.15789474) *
##        13) Fare< 20.8 59  26 1 (0.44067797 0.55932203) *
##          7) Pclass3< 0.5 121   9 1 (0.07438017 0.92561983) *

```

```
fancyRpartPlot(titanic_cart$finalModel)
```

Rattle 2026-Apr-05 15:26:28 ian_wang

```

titanic_cart_pred <- predict(titanic_cart, newdata = titanic_test)
titanic_cart_pred <- factor(titanic_cart_pred, levels = levels(titanic_test$Survived))

cm_titanic_cart <- confusionMatrix(
  titanic_cart_pred,
  titanic_test$Survived,
  positive = "1"
)

cm_titanic_cart$table

```

```

##           Reference
## Prediction  0  1
##           0 97 23
##           1  9 49

```

```
cm_titanic_cart$byClass[c("Sensitivity", "Specificity")]
```

```

## Sensitivity Specificity
##   0.6805556   0.9150943

```

```
cm_titanic_cart$overall["Accuracy"]
```

```

## Accuracy
## 0.8202247

```

Q7. Tune RF with tuneRF, fit final randomForest, plot importance, and compute test metrics

```
set.seed(123)
titanic_tune <- tuneRF(
  x = subset(titanic_train, select = -Survived),
  y = titanic_train$Survived,
  stepFactor = 1.5,
  improve = 0.01,
  trace = TRUE,
  plot = FALSE
)
```

```
## mtry = 2   OOB error = 20.34%
## Searching left ...
## Searching right ...
## mtry = 3   OOB error = 20.52%
## -0.009174312 0.01
```

```
titanic_tune
```

```
##      mtry OOBError
## 2.00B    2 0.2033582
## 3.00B    3 0.2052239
```

```
best_mtry_titanic <- titanic_tune[which.min(titanic_tune[, 2]), 1]
cat("Best mtry for Titanic RF:", best_mtry_titanic, "\n")
```

```
## Best mtry for Titanic RF: 2
```

```
set.seed(123)
titanic_rf <- randomForest(
  Survived ~ .,
  data = titanic_train,
  mtry = best_mtry_titanic,
  importance = TRUE
)
```

```
titanic_rf
```

```
##
```

```
## Call:
```

```
## randomForest(formula = Survived ~ ., data = titanic_train, mtry = best_mtry_titanic,
```

```
##               Type of random forest: classification
```

```
##               Number of trees: 500
```

```
## No. of variables tried at each split: 2
```

```
##
```

```
##               OOB estimate of error rate: 20.34%
```

```
## Confusion matrix:
```

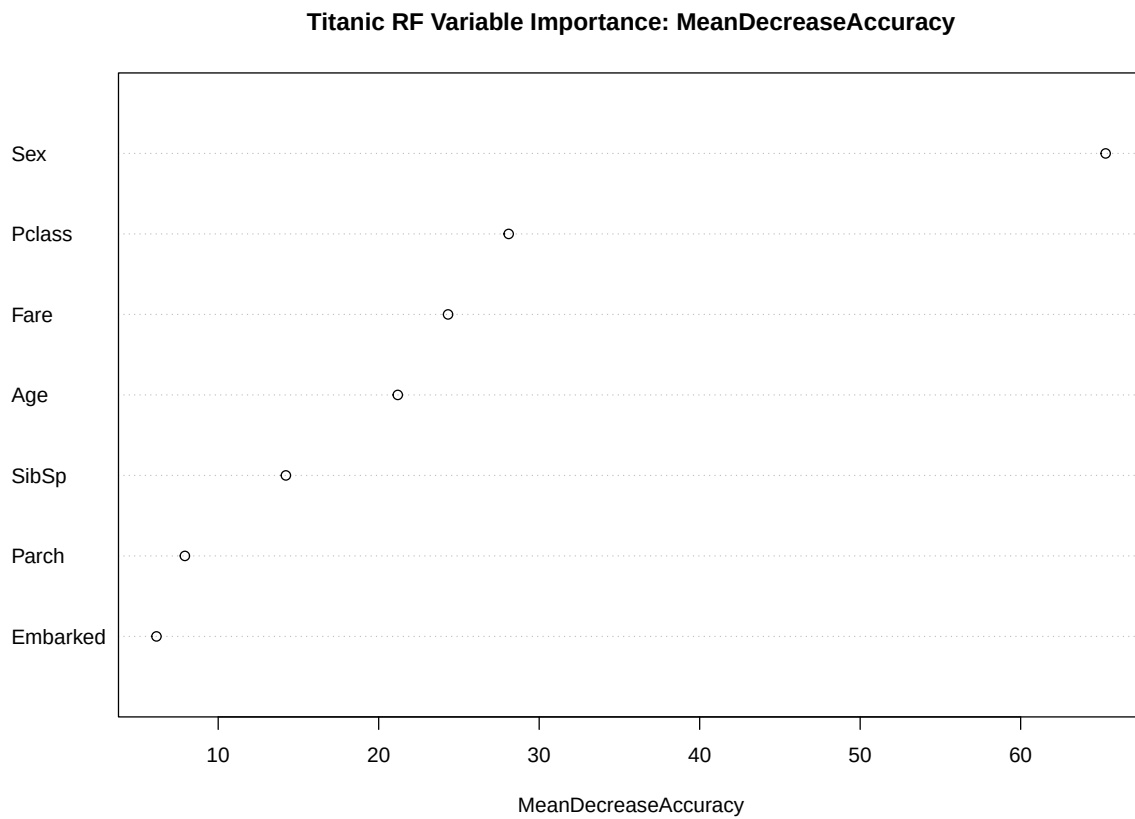
```
##      0   1 class.error
```

```
## 0 273  45  0.1415094
```

```
## 1  64 154  0.2935780
```

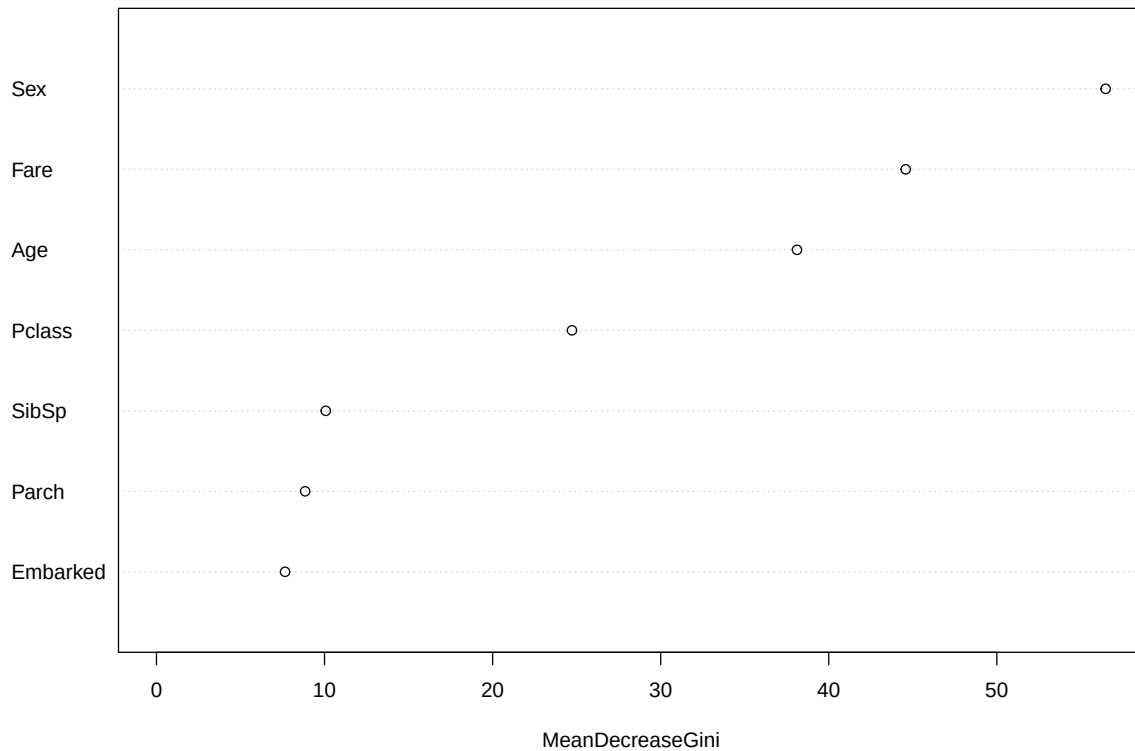
```
importance
```

```
randomForest::varImpPlot(
  titanic_rf,
  type = 1,
  main = "Titanic RF Variable Importance: MeanDecreaseAccuracy"
)
```



```
randomForest::varImpPlot(
  titanic_rf,
  type = 2,
  main = "Titanic RF Variable Importance: MeanDecreaseGini"
)
```

Titanic RF Variable Importance: MeanDecreaseGini



```
titanic_rf_pred <- predict(titanic_rf, newdata = titanic_test)
titanic_rf_pred <- factor(titanic_rf_pred, levels = levels(titanic_test$Survived))
```

```
cm_titanic_rf <- confusionMatrix(
  titanic_rf_pred,
  titanic_test$Survived,
  positive = "1"
)
```

```
cm_titanic_rf$table
```

```
##           Reference
## Prediction  0  1
##           0 89 15
##           1 17 57
```

```
cm_titanic_rf$byClass[c("Sensitivity", "Specificity")]
```

```
## Sensitivity Specificity
##  0.7916667  0.8396226
```

```
cm_titanic_rf$overall["Accuracy"]
```

```
## Accuracy
## 0.8202247
```

Q8. Compare CART and RF using test sensitivity, specificity, and accuracy

```
cart_sens <- cm_titanic_cart$byClass["Sensitivity"]
cart_spec <- cm_titanic_cart$byClass["Specificity"]
cart_acc  <- cm_titanic_cart$overall["Accuracy"]

rf_sens <- cm_titanic_rf$byClass["Sensitivity"]
rf_spec <- cm_titanic_rf$byClass["Specificity"]
rf_acc  <- cm_titanic_rf$overall["Accuracy"]

comparison <- data.frame(
  Model = c("CART", "Random Forest"),
  Sensitivity = c(cart_sens, rf_sens),
  Specificity = c(cart_spec, rf_spec),
  Accuracy = c(cart_acc, rf_acc)
)

comparison
```

```
##           Model Sensitivity Specificity  Accuracy
## 1           CART    0.6805556    0.9150943 0.8202247
## 2 Random Forest    0.7916667    0.8396226 0.8202247
```

The CART tree and the random forest achieved the same overall accuracy (0.8202), but their classification behavior was different. The random forest had noticeably higher sensitivity (0.7917 vs. 0.6806), which means it was better at correctly identifying passengers who survived. The CART tree had higher specificity (0.9151 vs. 0.8396), so it was better at correctly identifying passengers who did not survive. Overall, if the goal is to better detect survivors, the random forest performs better. If the goal is to reduce false positives for survival, the CART tree performs better.